

Exercices — `string` et fonctions

Ces exercices utilisent les notions présentées dans `string 00` à `string 05` : taille, comparaison, concaténation, accès avec `[]`, recherche, suppression, remplacement, extraction et quelques utilitaires supplémentaires.

L'objectif est aussi de choisir correctement le mode de passage des paramètres :

- `const string&` lorsque la fonction doit seulement consulter une chaîne;
- `string&` lorsque la fonction doit modifier la chaîne reçue;
- `string` par valeur lorsqu'il est volontairement utile de travailler sur une copie et de retourner le résultat.

1. Compter un caractère

Écrire la fonction :

```
int compterCaractere(const string& texte, char caractere);
```

La fonction retourne le nombre de fois où `caractere` apparaît dans `texte`.

Exemple :

```
compterCaractere("programmation", 'm') -> 2
```

La chaîne reçue ne doit pas être modifiée.

2. Remplacer un caractère

Écrire la fonction :

```
void remplacerCaractere(string& texte, char ancien, char nouveau);
```

Elle remplace toutes les occurrences de `ancien` par `nouveau` directement dans la chaîne reçue.

Exemple :

```
"banane", 'a', 'o' -> "bonone"
```

3. Retourner une copie en majuscules

Écrire la fonction :

```
string enMajuscules(string texte);
```

La fonction reçoit volontairement la chaîne **par valeur**, transforme sa copie en majuscules et retourne cette copie. La chaîne originale fournie par l'appelant ne doit donc pas être modifiée.

Exemple :

```
"Bonjour" -> "BONJOUR"
```

4. Inverser une chaîne

Écrire la fonction :

```
string inverser(const string& texte);
```

La fonction retourne une nouvelle chaîne contenant les caractères dans l'ordre inverse.

Exemple :

```
"abcde" -> "edcba"
```

Ne pas utiliser de fonction toute faite d'inversion.

5. Retirer tous les espaces

Écrire la fonction :

```
void retirerEspaces(string& texte);
```

Elle retire tous les caractères espace de la chaîne reçue.

Exemple :

```
"un deux trois" -> "undeuxTrois"
```

Cet exercice peut être réalisé avec `erase()` ou en reconstruisant progressivement la chaîne.

6. Extraire le nom et le prénom

Une personne est représentée sous la forme :

```
Mailloux, Mathieu
```

Écrire les deux fonctions suivantes :

```
string extraireNom(const string& personne);  
string extrairePrenom(const string& personne);
```

Utiliser `find()` pour repérer la virgule et `substr()` pour extraire les deux parties.

Le programme principal demande une ligne complète à l'utilisateur avec `getline()` puis affiche séparément le nom et le prénom.

7. Valider un code de cours

Un code de cours valide doit avoir exactement huit caractères et respecter le modèle suivant :

```
ABC-1234
```

Écrire :

```
bool codeCoursEstValide(const string& code);
```

Pour cet exercice, on considère valide un code dont les trois premiers caractères sont des lettres majuscules, le quatrième est '-' et les quatre derniers sont des chiffres. Vous pouvez vérifier les lettres et les chiffres directement avec des comparaisons de caractères.

Le programme principal demande des codes jusqu'à ce que l'utilisateur en fournisse un valide.

8. Compter une sous-chaine

Écrire :

```
int compterOccurrences(const string& texte, const string& recherche);
```

La fonction doit compter combien de fois `recherche` apparaît dans `texte`.

Exemple :

```
texte      = "un deux un trois un"  
recherche  = "un"  
résultat  = 3
```

Utiliser `find(recherche, positionDepart)` afin de poursuivre la recherche après chaque occurrence trouvée.

9. Remplacer toutes les occurrences d'un mot

Écrire :

```
void remplacerTout(string& texte,  
                  const string& recherche,  
                  const string& remplacement);
```

Exemple :

```
texte      = "chat chien chat"  
recherche  = "chat"  
remplacement = "tigre"  
  
résultat   = "tigre chien tigre"
```

Utiliser une combinaison de `find()` et `replace()`.

Attention à bien poursuivre la recherche après le texte qui vient d'être inséré.

10. Palindrome

Un palindrome se lit de la même manière de gauche à droite et de droite à gauche.

Écrire :

```
bool estPalindrome(const string& texte);
```

Première version : tenir compte exactement des caractères et de la casse.

```
"radar" -> true  
"Radar" -> false
```

Deuxième version : écrire une fonction supplémentaire :

```
string normaliser(string texte);
```

Elle retourne une copie sans espaces et en majuscules. `estPalindrome()` peut ensuite utiliser cette fonction afin que :

```
"Esope reste ici et se repose" -> true
```

11. Censurer un mot

Écrire :

```
string censurer(const string& texte, const string& mot);
```

La fonction retourne une copie du texte dans laquelle chaque occurrence du mot est remplacée par le même nombre de '*'.

Exemple :

```
censurer("un chat et un autre chat", "chat")
```

```
-> "un **** et un autre ****"
```

Décomposer le problème : une petite fonction peut d'abord fabriquer une chaîne contenant le bon nombre d'astérisques.

12. Mini-programme — Analyse d'une phrase

Le programme demande une phrase complète avec `getline()` puis utilise plusieurs fonctions :

```
int compterCaractere(const string& texte, char caractere);  
int compterEspaces(const string& texte);  
string enMajuscules(string texte);  
string inverser(const string& texte);
```

Il affiche ensuite :

```
Phrase : Bonjour tout le monde  
Longueur : 21  
Nombre d'espaces : 3  
Nombre de o : 4  
Majuscules : BONJOUR TOUT LE MONDE  
Inversee : ednom el tuot ruojnoB
```

Le `main()` doit surtout coordonner les appels de fonctions; les traitements doivent être réalisés dans les fonctions.

13. Mini-programme — Générateur d'identifiant

Le programme demande :

```
Prénom : Mathieu  
Nom : Mailloux  
Année de naissance : 1980
```

Créer plusieurs petites fonctions afin de produire un identifiant sous la forme :

```
mmailoux80
```

Découpage suggéré :

```
string mettreEnMinuscules(string texte);  
string deuxDerniersCaracteres(const string& texte);  
string creerIdentifiant(const string& prenom,  
                        const string& nom,  
                        const string& annee);
```

Le programme doit fonctionner avec des noms et prénoms de longueurs différentes.

14. Défi — Réduire les espaces multiples

Écrire :

```
void normaliserEspaces(string& texte);
```

La fonction transforme toute suite de plusieurs espaces en un seul espace et retire les espaces au début et à la fin.

Exemple :

```
"  ceci  est  un test  "  
  
-> "ceci est un test"
```

Ce défi demande surtout de bien manipuler les indices pendant que la taille de la `string` change.